# 1 UNDERSTANDING ALGORITHMS

**GENERAL VOCABULARY**

**construct** a command to control the order/flow in which instructions are executed (e.g. sequences, selection, repetition)

It is important to be able to **construct** algorithms and be able to read them and follow their logic in solving particular problems.

**LEARNING OBJECTIVES**

- Understand what an algorithm is
- Understand what algorithms are used for
- Interpret algorithms as flowcharts, pseudocode and written descriptions
- Use and describe the purpose of arithmetic operators
- Understand how to code an algorithm in a high-level language

## AN EXAMPLE OF AN ALGORITHM

**SUBJECT VOCABULARY**

**unambiguous** this means that the instructions cannot be misunderstood. Simply saying 'turn' would be ambiguous (i.e. unclear) because you could turn left or right. All instructions given to a computer must be unambiguous or it won't know what to do

**sequence** an ordered set of instructions

**algorithm** a precise method for solving a problem

An interactive map is a useful way to find a route between two locations. Figure 1.1 shows a route between two cities that was calculated by a mapping program.

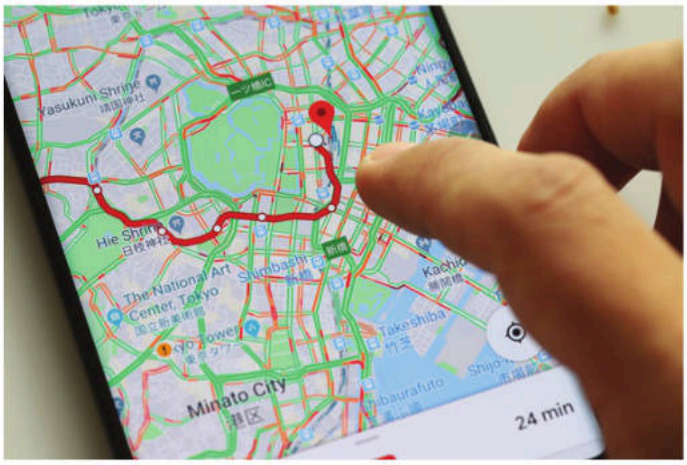
The route on this interactive map has been calculated using an algorithm.

- It is **unambiguous** in telling the driver exactly what to do, like 'turn left', 'turn right' or 'go straight'.
- It is a **sequence** of steps.
- It can be used again and will always provide the same result.
- It provides a solution to a problem, in this case, how to get from Beijing to Shanghai.

A solution to a problem with these characteristics is called an **algorithm**. Most problems have more than one solution, so different algorithms can be created for the same problem.

**DID YOU KNOW?**

The computer program that created the algorithm to map travel from Beijing to Shanghai was following an algorithm of its own – an algorithm ordering it how to create another algorithm!




▶ **Figure 1.1** A route calculated by a mapping program

## SUCCESSFUL ALGORITHMS

There are three points to consider when deciding whether an algorithm is successful or not.

**GENERAL VOCABULARY**

**outcome** the final result of an action

**consistency** not changing; always the same

**atlas** a book of maps

- Accuracy – it must lead to the expected **outcome** (e.g. create a route from Beijing to Shanghai).
- **Consistency** – it must produce the same result each time it is run.
- Efficiency – it must solve the problem in the shortest possible time, using as few computer resources as possible. In this example, the mapping software is replacing a manual method. If it were no faster than looking in an **atlas**, then it would not be an improvement on the older method. Later in the unit there is a section on algorithms that are used to sort and search data. Some of these algorithms are more efficient than others and will sort the data far more quickly.

## THE RELATIONSHIP BETWEEN ALGORITHMS AND PROGRAMS

Algorithms and programs are closely related, but they are not the same. An algorithm is a detailed design for a solution; a program is when that design is implemented.

**SUBJECT VOCABULARY**

**high-level programming language** a programming language that is similar to natural human language

This unit is all about algorithms. We look at how algorithms are implemented in **high-level programming languages** in Unit 2.

## DISPLAYING AN ALGORITHM

We carry out many everyday tasks using algorithms because we are following a set of instructions to achieve an expected result, for example, making a cup of coffee. If we have performed the task many times before, we usually carry out the instructions without thinking. But if we are doing something unfamiliar, such as putting together a flat-pack chest of drawers, then we follow the instructions very carefully.

An algorithm can be expressed in different ways.

### WRITTEN DESCRIPTIONS

A written description is the simplest way of expressing an algorithm. Here is an algorithm describing the everyday task of making a cup of instant coffee:



**ALGORITHM FOR MAKING A CUP OF COFFEE**

Fill kettle with water.
Turn on kettle.
Place coffee in cup.
Wait for water to boil.
Pour water into cup.
Add milk and sugar.
Stir.

# 4 DECOMPOSITION AND ABSTRACTION

When computer scientists attempt to solve problems by producing algorithms and coding them into programs, they approach the problems in a particular way. This method has been given the name 'computational thinking'. Before they create a structured solution or algorithm and code it into a program, they must define and analyse the problems. This section will introduce two of the **thought processes** they use – decomposition and abstraction.

**GENERAL VOCABULARY**

**thought process** the act of using your mind to consider or think about something

**LEARNING OBJECTIVES**

- Analyse a problem, investigate requirements (inputs, outputs, processing, initialisation) and design solutions
- Decompose a problem into smaller sub-problems
- Understand how abstraction can be used effectively to model aspects of the real world
- Program abstractions of real-world examples

## PROBLEM SOLVING

**SUBJECT VOCABULARY**

**computational thinking** the thought processes involved in formulating problems and their solutions so that the solutions are represented in a form that can be effectively carried out by a computer

**decomposition** breaking a problem down into smaller, more manageable parts, which are then easier to solve

**abstraction** the process of removing or hiding unnecessary detail so that only the important points remain

The tasks of a computer scientist include defining and analysing problems; creating structured solutions – algorithms; and coding the solutions into a form that can be implemented by a computer. These tasks are part of what is known as **computational thinking**.

One of the skills required for computational thinking is algorithm design (which we've covered in detail in this unit). If there is a fault in the algorithm design, then the program will not work, however good a coder you are. Two other skills are **decomposition** and **abstraction**.

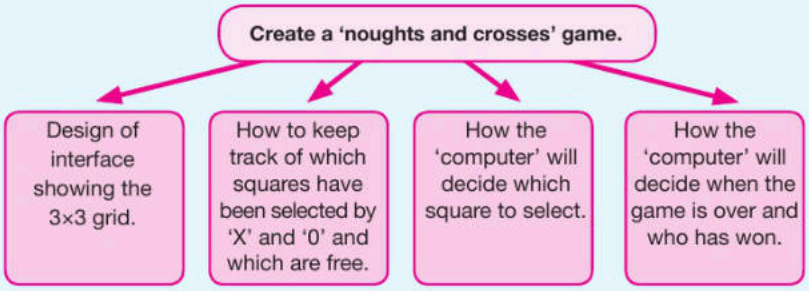
## DECOMPOSITION

Decomposition is usually the first step in the problem-solving process. Once a problem has been broken down and the sub-problems have been identified, algorithms can be developed to solve each of them.

Decomposition means that sub-problems can be worked on by different teams at the same time. As smaller algorithms are developed for each sub-problem, it is easier to spot and correct errors. When the algorithm is developed into a program, code can be used again.

## WORKED EXAMPLE

A student has been set the task of creating a computer version of the game 'noughts and crosses' (also known as 'tic-tac-toe') where a user plays against the computer.




▶ **Figure 1.12** Sub-problems to be solved to create a noughts and crosses computer program

The diagram shows some of the sub-problems that must be solved in order to solve the complete problem and create a version of the game.

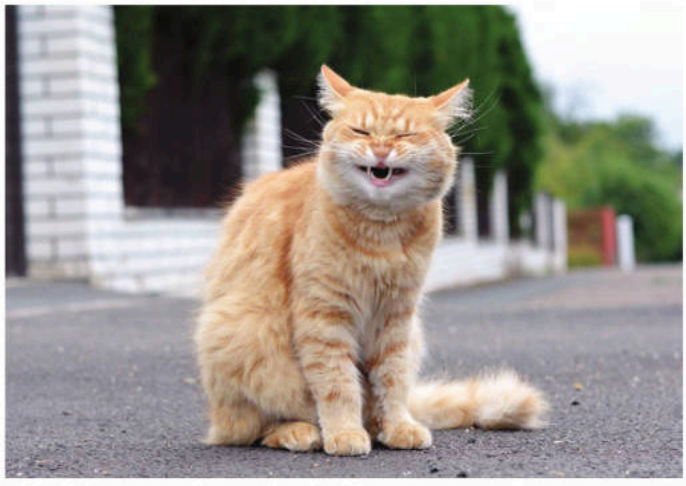
## ABSTRACTION

We use abstraction all the time in our daily lives. We **abstract** the essential features of something so that we can understand what people are trying to communicate.

**GENERAL VOCABULARY**

**abstract** to remove something from somewhere

Somebody might say, 'I was walking down the street when I saw a cat'. You immediately understand what they mean by 'street' – probably a road with a pavement and houses or shops along the side of it. Similarly, you can picture the cat – a small animal with fur, four legs and a tail. An animal that is basically 'cattish'. You have extracted the basic properties of animals called cats so that you can recognise one when you see one, or imagine one when somebody talks about a cat.



▲ Is this the street and cat you imagined?

What you imagine is very unlikely to match the actual street and cat that the person experienced. But, because of our ability to abstract, the person did not

have to go into unnecessary detail about exactly where they were and what they saw. They wouldn't get very far with the story if they did.

When we create algorithms, we abstract the basic details of the problem and represent them in a way that a computer is able to process.

## WORKED EXAMPLE

Yasmin is designing a computer version of a game in which users have to throw a die to find out their number of moves.

In the computer game, the users can't have an actual die, so she will have to design a 'pretend' or virtual die that behaves in exactly the same way as a real-life die.

Yasmin will have to use her powers of abstraction to work out the essential features of a die and then represent them in computer code.

To represent the die, she will have to create a routine that will select a random number from 1 to 6 because that's what a die does.

Yasmin has used abstraction to **model** a real-life event.

**GENERAL VOCABULARY**

**model** to make a simple version of something to show how it works

## LEVELS OF ABSTRACTION

There are different levels or types of abstraction. The higher the level of abstraction, the less detail is required. We use abstraction all the time in accomplishing everyday tasks.

When programmers write the 'print' command they do not have to bother about all of the details of how this will be accomplished. They are removed from them. They are at a certain level of abstraction.

A driver turning the ignition key to start a car does not have to understand how the engine works or how the spark to ignite the petrol is generated. It just happens and they can simply drive the car. That is abstraction.



▲ A game of noughts and crosses

## AN EXAMPLE – NOUGHTS AND CROSSES

Figure 1.12 showed some of the sub-problems that the problem of creating a noughts and crosses game could be divided into. The following could be written at a high level of abstraction.

- The computer goes first. Then the user. This continues until either one wins, or all of the squares have been used.

Immediately a pattern can be recognised – a loop will be needed.

**Inputs and outputs**

The following inputs from the user will be needed.

- Start the game.
- Entries for the user.
- Select a new game or finish.

The following outputs will be needed.

- A message to inform the user when it is their turn.
- A message to inform the user if they try to select a square that has already been used.